

UNIVERSIDADE FEDERAL DE MINAS GERAIS
Departamento de Engenharia Elétrica
Programa de Graduação em Engenharia de Sistemas

Samuel Lima Horta
2023060561

Felipe Lucca Taumaturgo de Oliveira
2023060723

Turma N3

Projeto Final - Lab. SD
Etapa 2 – Arquitetura RTL

Belo Horizonte
16 de Junho 2026

0.1 Arquitetura RTL e Máquina de Estados Finitos

0.2 Máquina de Estados Finitos (FSM)

A Unidade de Controle do sistema foi implementada por meio de uma Máquina de Estados Finitos, adotada para organizar de forma estruturada as operações de entrada, saída e atualização das informações do estacionamento. A FSM controla os eventos gerados pelos dispositivos de entrada e utiliza os sinais de status do Caminho de Dados para determinar a próxima ação a ser executada.

A máquina de estados foi projetada com nove estados distintos: *Reset*, *Espera*, *Entrada*, *Verifica-Capacidade*, *Registra-Entrada*, *Saída*, *Registra-Saída*, *Lotado* e *Atualiza-Saídas*. Essa divisão foi escolhida para separar as diferentes etapas de operação do sistema, tornando o fluxo de controle mais organizado e facilitando sua implementação em hardware. O estado *Reset* realiza a inicialização dos registradores e prepara o sistema para operação. Em seguida, o estado *Espera* permanece monitorando os sinais de entrada até que seja detectada uma solicitação de entrada ou saída de veículo.

Quando uma solicitação é detectada, a FSM verifica a disponibilidade de vagas por meio dos sinais gerados pelo comparador e executa a operação correspondente de entrada ou saída. Após a atualização dos registradores, o estado é atualizados e os displays e indicadores luminosos do sistema indicam com sinais relacionados sobre estado de lotação atual do estacionamento, retornando posteriormente ao estado *Espera* para aguardar novos eventos.

A Figura 1 apresenta o diagrama completo da Máquina de Estados Finitos desenvolvida para o projeto.

0.3 Arquitetura RTL

A arquitetura RTL do sistema foi organizada em três blocos principais: Unidade de Controle, Caminho de Dados e Interface de Saída. Essa divisão foi adotada por facilitar a separação entre as funções de controle e as operações de processamento de dados, tornando o projeto mais modular e adequado para implementação em hardware.

A Unidade de Controle foi implementada por meio de uma Máquina de Estados Finitos, escolhida para coordenar as operações do sistema de forma organizada e previsível. Já o Bloco Operacional reúne os registradores e componentes combinacionais responsáveis pelo armazenamento e processamento das informações do estacionamento, como a quantidade de veículos presentes, a capacidade máxima configurada e o número de vagas disponíveis.

A comunicação entre os blocos ocorre por meio de sinais de controle enviados pela

FSM ao Caminho de Dados e sinais de status retornados pelo Caminho de Dados à FSM. Essa troca de informações permite que as decisões de controle sejam tomadas com base na situação atual do estacionamento, como disponibilidade de vagas, lotação máxima ou ocorrência de eventos temporizados.

A Figura 2 apresenta a arquitetura RTL desenvolvida para o projeto.

0.3.1 Sinais de Controle

Os sinais de controle são gerados pela Unidade de Controle e utilizados para coordenar as operações executadas no Caminho de Dados. Esses sinais determinam quando os registradores devem ser atualizados e quais operações devem ser realizadas em cada estado da FSM, garantindo que o sistema execute corretamente os processos de entrada, saída e atualização das informações do estacionamento.

0.3.2 Sinais de Status

Os sinais de status são produzidos pelo Caminho de Dados e utilizados pela FSM para acompanhar a condição atual do sistema. Esses sinais informam situações como disponibilidade de vagas, lotação máxima e término de temporizações, permitindo que a Unidade de Controle tome decisões adequadas em cada etapa de operação.

0.4 Lista de Componentes

Nesta seção apresentamos os principais componentes utilizados na arquitetura RTL do Sistema de Estacionamento Inteligente, juntamente com suas funções, sinais de entrada, sinais de saída e larguras. A seleção desses componentes foi realizada com base nas operações identificadas durante a especificação funcional e na necessidade de separar as funções de controle, armazenamento e processamento de dados de acordo com a metodologia RTL adotada no projeto.

Tabela 1 – Sinais de controle gerados pela FSM

Sinal	Função
inc_veiculos	Incrementa o contador de veículos.
dec_veiculos	Decrementa o contador de veículos.
load_historico	Atualiza o registrador de histórico.
atualiza_saidas	Atualiza os displays e LEDs do sistema.
reset_regs	Reinicializa os registradores do sistema.

Tabela 2 – Sinais de status enviados ao controle

Sinal	Função
vaga_disponivel	Indica a existência de vagas livres no estacionamento.
estacionamento_lotado	Indica que a capacidade máxima foi atingida.
estacionamento_vazio	Indica ausência de veículos no estacionamento.
atualiza_saidas_hex	Envia sinal para atualizar as saidas do display HEX.

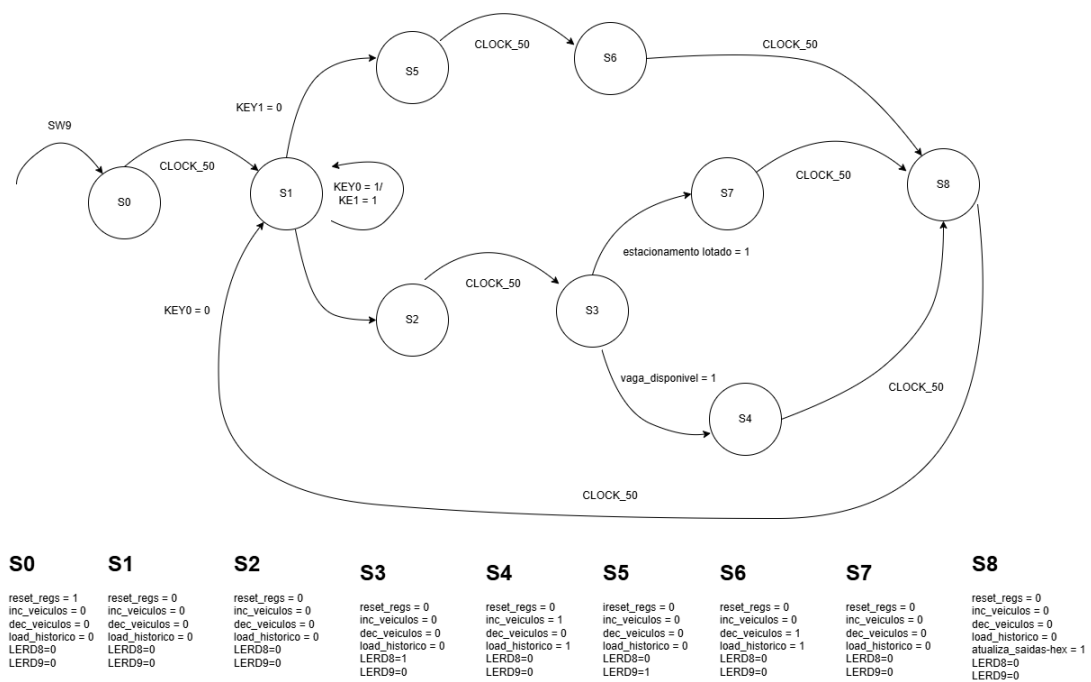


Figura 1 – Diagrama da FSM.

Tabela 3 – Componentes da Arquitetura RTL

Componente	Função	Entradas	Saídas
FSM	Controlar a sequência de execução do sistema e gerar sinais de controle.	CLOCK_50, KEY0, KEY1, SW[3:0], SW9(reset)	inc_veiculos, dec_veiculos, load_historico, reset_regs
Registrador de Veículos	Armazenar a quantidade atual de veículos.	clk, reset, inc_veiculos, dec_veiculos,	veiculos_atual[3:0]
Registrador de Capacidade	Armazenar a capacidade máxima configurada.	SW[3:0], clk, reset	capacidade_atual[3:0]
Registrador de Vagas Livres	Armazenar a quantidade de vagas disponíveis.	vagas_livres_entrada[3:0], clk, reset	vagas_livres[3:0]
Registrador de Histórico	Registrar as últimas movimentações realizadas.	load_historico[7:0], clk, reset	-
Somador	Realizar a operação Veículos + 1.	veiculos[3:0]	veiculos+1 - [3:0]
Subtrator	Realizar a operação Veículos - 1.	veiculos[3:0]	veiculos-1 - [3:0]
Comparador	Comparar a quantidade de veículos com a capacidade máxima.	veiculos_atual[3:0], capacidade_atual[3:0]	vaga_disponivel, estacionamento_lotado, estacionamento_vazio, saida_led0, saida_led1, saida_led2
Calculador de Vagas Livres	Calcular vagas livres disponíveis.	capacidade_atual[3:0], veiculos_atual[3:0]	vagas_livres[3:0]
Decodificador HEX	Converter valores binários para displays de sete segmentos.	veiculos_atual[3:0], vagas_livres[3:0]	segmentos_hex[6:0]
Interface de Saída	Atualizar displays e LEDs do sistema.	Dados do Bloco operacional	HEX0, HEX1, LEDR0, LEDR1, LEDR2, LEDR8, LEDR9

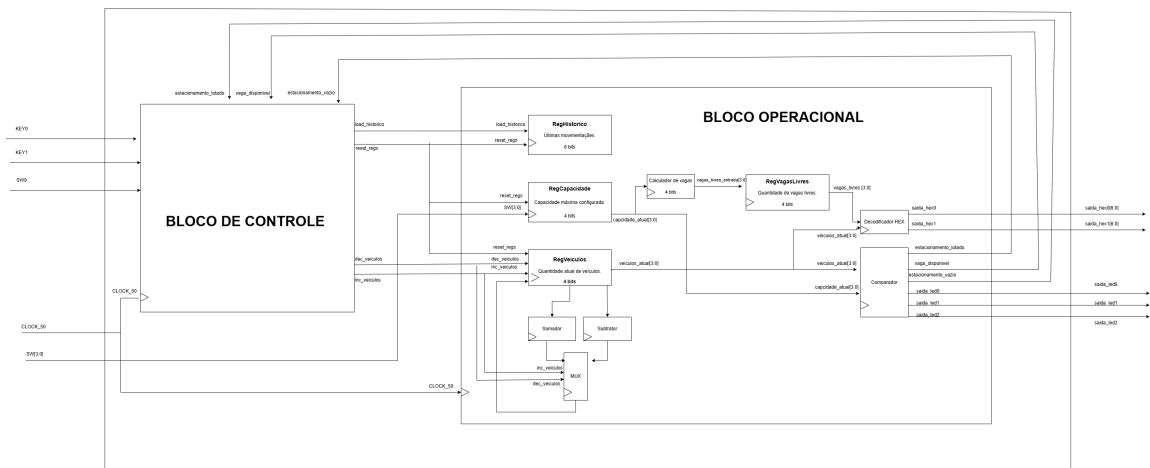


Figura 2 – Arquitetura RTL do Sistema.